Fan Inventory Implementation Plan

Goal and assumptions

Add a first-class Fan Inventory that users can create inside a directory, populate and edit, review in summary views, and persist with the existing inventory/settings architecture. This plan treats “Fan Inventory” as a new inventory record, distinct from the existing FSAT assessment and fan calculators. The precise data fields, hierarchy, calculation responsibilities, and FSAT connection are open product/domain decisions; this plan does not prescribe engineering values or imply a new calculation API.

Read the companion inventory architecture review before implementation.

Phase 1: Set the product and domain contract

1. Confirm the MVP workflows and user outcomes: create, organize, edit, validate, summarize, copy, export/import, and delete.
2. Decide whether fan assets belong in departments, systems, or another domain-specific grouping, and whether groups need their own properties.
3. Define a fan item’s stable identifiers, required and optional information groups, defaults, validation rules, conditional fields, and summary/filter needs.
4. Decide how Fan Inventory relates to FSAT: no link in the MVP, one fan per assessment, or another relationship. Specify ownership, navigation, data synchronization, disconnect/copy behavior, and source-of-truth rules if integration is in scope.
5. Establish the units and percentage conventions for every numeric field, supported settings conversion behavior, and compatibility expectations for backups/imported records.
6. Set explicit MVP boundaries for graphs, reports, catalog lookup, motor/drive links, CO_2 estimates, and other optional features.

Exit criteria: reviewed domain model and interaction scope; integration and persistence compatibility decisions recorded before implementation.

Phase 2: Add the feature model and core domain behavior

1. Create `src/app/fan-inventory/` following the existing feature-folder convention.
2. Add fan inventory interfaces that match the decided hierarchy and keep item properties in cohesive groups. Avoid reusing pump or motor interfaces solely because fields look similar.
3. Define a feature service responsible for initial inventory data, group/item creation and removal, validation, summary filtering, and feature-view state.
4. Make initial group/item values and display defaults explicit, including behavior for an empty inventory and invalid or incomplete items.

5. Add form/catalog services as needed to centralize field setup and validation, and have validity depend on the agreed required data rather than the presence of unrelated optional fields.
6. Add a dedicated unit-conversion service if persisted data changes representation with user units. Cover each numeric field in both directions, preserve null/undefined values, and document fields that are not converted.

Exit criteria: typed model, deterministic initialization, validation ownership, and unit behavior are testable without relying on the UI.

Phase 3: Build setup, navigation, and summary flows

1. Create the feature entry component, setup components, banner/help experience, and feature module. Keep persistence and cross-feature coordination out of individual field controls.
2. Define route constants and ordered setup/summary views. Make direct navigation, refresh, back/forward, and validity-gated transitions behave consistently.
3. Build the MVP overview/table summary using shared exportable-result components where suitable. Add graphs or additional reports only if included in Phase 1.
4. Add filters and display-option settings only for supported MVP use cases; ensure filtering does not mutate source inventory data.
5. Reuse shared form, modal, settings, help, and unit-label facilities while retaining fan-specific domain logic in this module.

Exit criteria: a user can complete the agreed fan setup path, reach the applicable summaries, and navigate directly to supported routes.

Phase 4: Connect the feature to app-wide inventory flows

Update the shared entry points identified in the architecture review:

1. Add the optional fan data property to `InventoryItem`, and add a stable inventory type discriminator such as `fanInventory`.
2. Add a `getNewFanInventoryItem` factory in `dashboard/inventory.service.ts` using the feature service's initialization.
3. Extend creation UI and type/name selection in `dashboard/create-inventory/`.
4. Register the feature route and module in core routing/module wiring.
5. Add fan inventory handling to directory cards, sidebar navigation, selection, rename/move, and directory item routing.
6. Verify delete removes associated settings and any required fan-specific connected records.
7. Extend copy handling. Remove or deliberately remap connections so copied data does not silently continue to reference the source inventory or assessment.

8. Verify generic import/export can carry the fan data and settings, clear old storage IDs, restore a new inventory ID, and retain only intended in-record identifiers/links. Add normalization or a schema migration if supported import versions require it.
9. If a fan-to-FSAT or fan-to-motor integration is approved, extend shared integration types/services and assessment entry points in a separate, explicit slice. Keep any suite calculation calls behind `src/app/tools-suite-api/`.

Exit criteria: fan inventories behave like other inventory records in directory lifecycle, persistence, settings, copy, backup/import, and export paths.

Phase 5: Verify behavior and compatibility

Add tests for:

- Initial data, item/group operations, required/conditional validation, filtering, and any numeric conversions.
- Route defaults, route order, invalid-state gating, and direct tab navigation.
- Inventory creation and persisted type/data shape; save/reload and associated settings.
- Directory display/routing, moving, duplication, deletion, and connection cleanup.
- Single-item and directory-level import/export round trips, including changed IDs and legacy or incomplete data.
- Any approved FSAT/motor connection lifecycle, including source updates, restoration, disconnect, and copied records.

Then run the narrow tests first and the repository's existing Angular validation as needed: `npm test`, `npx tsc -p src/tsconfig.app.json --noEmit`, and `npm run build`. Do not add a separate database store unless a documented data requirement cannot fit the shared inventory store. Review Electron/backup workflows if the persisted schema or backup behavior changes.

Delivery sequence and acceptance

Prefer independent increments: (1) product/domain contract, (2) model/service and focused tests, (3) setup/routes/summary, (4) dashboard/persistence integrations, (5) import/export and optional links, (6) full verification and documentation. Keep FSAT integration and advanced graphs out of the MVP unless confirmed in Phase 1.

The work is ready when users can create a Fan Inventory from a directory, edit and save it, return to it after reload, see its supported summary, and use shared move/copy/delete/import/export flows without losing or misdirecting data. All units, validity rules, route behavior, and integration decisions must be verified by tests and documented alongside the feature.